

Отчет по лабораторной работе №1
по курсу «Алгоритмы и структуры
данных»

Тема: Сортировка слиянием. Метод
декомпозиции

Выполнил:

Афанасьев Д.М.

К3140

Проверил:

Афанасьев А.В.

Санкт-Петербург

2024 г.

Содержание

Содержание.....	2
Задание 1.....	3
Задание 2.....	5
Задание 3.....	7
Задание 4.....	9
Задание 5.....	11
Задание 6.....	14
Вывод:.....	16

Задание 1

Используя псевдокод процедур Merge и Merge-sort из презентации к Лекции 2 (страницы 6-7), напишите программу сортировки слиянием на Python и проверьте сортировку, создав несколько случайных массивов

```
def merge(array: list[int], left: int, center: int,
          right: int) -> list[int]:

    left_array = array[left:center]
    right_array = array[center:right]
    id_left = 0
    id_right = 0

    left_array.append(2**62 - 1)
    right_array.append(2**62 - 1)

    for key in range(left, right):
        if left_array[id_left] <=
right_array[id_right]:
            array[key] = left_array[id_left]
            id_left += 1
```

```

        else:
            array[key] = right_array[id_right]
            id_right += 1
    return array

def merge_sort(array: list[int], left, right) ->
list[int]:
    if right - left != 1:
        center = (left+right)//2

        merge_sort(array, left, center)
        merge_sort(array, center, right)

        merge(array, left, center, right)

    return array

```

- NormVid преобразует массив в строку, которая должна получиться на выходе
- переписан код из псевдокода, немного изменены индексы

Тесты

Использовался unittest

ЛИСТИНГ ТЕСТОВ

```
import memory_profiler
import random
import time
import unittest

from lab2.task1.src.main import merge_sort

def normVid(array: list) -> str:
    s = ""
    for i in range(len(array) - 1):
        s += str(array[i]) + " "
    s += str(array[-1])
    return s

class MergeSortTestCase(unittest.TestCase):

    def test_merge(self):
        file = open(
            "C:\\Users\\User\\PycharmProjects\\algorithms-and-
            data-structures\\lab2\\task1\\src\\input.txt").rea
        dlines()[
```

```

        1]

        array = list(map(int, file.split()))

        memory_before =
memory_profiler.memory_usage()[0]

        sorted_array = merge_sort(array, 0,
len(array))

        tmp = time.time()

        self.assertEqual(sorted_array,
sorted(array))

        tmp = time.time() - tmp

        memory_after =
memory_profiler.memory_usage()[0]

        print("Время выполнения: ", tmp)
        print("Использование памяти: ",
memory_after - memory_before, "МБ")

        with
open("C:\\Users\\User\\PycharmProjects\\algorithms
-and-data-structures\\lab2\\task1\\src\\output.txt
",

            "w") as file:
            file.write(normVid(sorted_array))

```

```

def test_merge_1000(self):
    N = 1000
    array = [i for i in range(N, 0, -1)]

    memory_before =
memory_profiler.memory_usage()[0]

    sorted_array = merge_sort(array, 0,
len(array))

    tmp = time.time()
    self.assertEqual(sorted_array,
sorted(array))
    tmp = time.time() - tmp

    memory_after =
memory_profiler.memory_usage()[0]

    print()
    print(N)
    print("Время выполнения: ", tmp)
    print("Использование памяти: ",
memory_after - memory_before, "МБ")

def test_merge_10000(self):
    N = 10000

```

```

        array = [i for i in range(N, 0, -1)]

        memory_before =
memory_profiler.memory_usage()[0]

        sorted_array = merge_sort(array, 0,
len(array))

        tmp = time.time()
        self.assertEqual(sorted_array,
sorted(array))
        tmp = time.time() - tmp

        memory_after =
memory_profiler.memory_usage()[0]

        print()
        print(N)
        print("Время выполнения: ", tmp)
        print("Использование памяти: ",
memory_after - memory_before, "МБ")

    def test_merge_100000(self):
        N = 100000
        array = [i for i in range(N, 0, -1)]

```



```

        memory_before =
memory_profiler.memory_usage()[0]

        sorted_array = merge_sort(array, 0,
len(array))

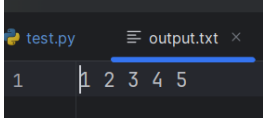
        tmp = time.time()
        self.assertEqual(sorted_array,
sorted(array))
        tmp = time.time() - tmp

        memory_after =
memory_profiler.memory_usage()[0]

        print()
        print(N)
        print("Время выполнения: ", tmp)
        print("Использование памяти: ",
memory_after - memory_before, "МБ")

if __name__ == '__main__':
    unittest.main()

```

длина массива	Время	Память	Вывод
Пример из задачи	0	0.07 MB	
1000	0	0.00390625 MB	[1, 2, 3, 4, 5, 6, 7, 8, 9, 10...]
10**4	0	0.15625 MB	[1, 2, 3, 4, 5, 6, 7, 8, 9, 10...]
10**5	0.0009	0.105 MB	[1, 2, 3, 4, 5, 6, 7, 8, 9, 10...]

-

Задание 1.1

3. Перепишите процедуру Merge так, чтобы в ней не использовались сигнальные значения. Сигналом к остановке должен служить тот факт, что все элементы массива L или R скопированы обратно в массив A, после чего в этот массив копируются элементы, оставшиеся в непустом массиве.

```
def merge(array: list[int], left: int, center: int,
          right: int) -> list[int]:

    left_array = array[left:center]
    right_array = array[center:right]

    for key in range(left, right):
```

```

    if len(left_array) == 0:
        array[key] = right_array.pop(0)
        continue
    if len(right_array) == 0:
        array[key] = left_array.pop(0)
        continue

    if left_array[0] <= right_array[0]:
        array[key] = left_array.pop(0)
    else:
        array[key] = right_array.pop(0)
return array

def merge_sort(array: list[int], left, right) ->
list[int]:
    if right - left != 1:
        center = (left+right)//2

        merge_sort(array, left, center)
        merge_sort(array, center, right)

    merge(array, left, center, right)

```

```
return array
```

добавлено новое условие

ввод	Время	Память	Вывод
10**5	0.0009	0.105 МВ	[1, 2, 3, 4, 5, 6, 7, 8, 9, 10...]

-

-

Задание 2

Дан массив целых чисел. Ваша задача — отсортировать его в порядке неубывания с помощью сортировки слиянием. Чтобы убедиться, что Вы действительно используете сортировку слиянием, мы просим Вас, после каждого осуществленного слияния (то есть, когда соответствующий подмассив уже отсортирован!), выводить индексы граничных элементов и их значения.

```
from random import random, randint

def merge(array: list[int], left: int, center: int,
          right: int) -> list[int]:
```

```

left_array = array[left:center]
right_array = array[center:right]
id_left = 0
id_right = 0

left_array.append(2 ** 62 - 1)
right_array.append(2 ** 62 - 1)

for key in range(left, right):
    if left_array[id_left] <=
right_array[id_right]:
        array[key] = left_array[id_left]
        id_left += 1
    else:
        array[key] = right_array[id_right]
        id_right += 1
with open("output.txt", "a") as file:
    file.write(f'{left + 1} {right} {array[left]}
{array[right - 1]}\n')
    file.close()
return array

def merge_sort(array: list[int], left, right) ->
list[int]:
    if right - left != 1:
        center = (left + right) // 2

```

```

        merge_sort(array, left, center)
        merge_sort(array, center, right)

    merge(array, left, center, right)

    return array

N = 10 ** 5
# array = [randint(1, 10**9) for i in range(N)]
array = [1, 8, 2, 1, 4, 7, 3, 2, 3, 6]
with open("output.txt", "w") as file:
    file.write("")
    file.close()

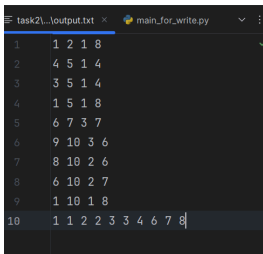
def normVid(array: list) -> str:
    s = ""
    for i in range(len(array) - 1):
        s += str(array[i]) + " "
    s += str(array[-1])
    return s

a = merge_sort(array, 0, len(array))
with open("output.txt", "a") as file:

```

```
file.write(normVid(a))
file.close()
```

- индексы - это left, center, right
- записываем в файл
- отличается от вывода в примере

ввод	Время	Память	Вывод
пример	0	0.00390625 МБ	
10**4	0	0.00390625 МБ	[1, 2, 3, 4, 5, 6, 7, 8, 9, 10...]
10**5	0.00104	0.105 МБ	[1, 2, 3, 4, 5, 6, 7, 8, 9, 10...]

- главное, что совпадает первый и последний вывод, промежуточные подмассивы могут быть различны, а значит и их индексы тоже
-

Задание 3

Инверсией в последовательности чисел A называется такая ситуация, когда $i < j$, а $A_i > A_j$. Количество инверсий в последовательности в некотором роде определяет, насколько близка данная

последовательность к отсортированной. Например, в отсортированном массиве число инверсий равно 0, а в массиве, отсортированном наоборот - каждые два элемента будут составлять инверсию (всего $n(n - 1)/2$). Дан массив целых чисел. Ваша задача — подсчитать число инверсий в нем. Подсказка: чтобы сделать это быстрее, можно воспользоваться модификацией сортировки слиянием

```
def merge(array: list[int], left: int, center: int,
          right: int) -> int:
    left_array = array[left:center]
    right_array = array[center:right]
    id_left = 0
    id_right = 0

    left_array.append(2 ** 62 - 1)
    right_array.append(2 ** 62 - 1)

    res = 0

    for key in range(left, right):
        if left_array[id_left] <=
right_array[id_right]:
            array[key] = left_array[id_left]
            id_left += 1
            res += (key - center)
        else:
            res += (center - key)
```



```

        array[key] = right_array[id_right]
        id_right += 1

    return res

def merge_sort(array: list[int], left, right) ->
    int:
    if right - left != 1:
        center = (left + right) // 2

        res = merge_sort(array, left, center)
        res += merge_sort(array, center, right)
        res += merge(array, left, center, right)
    return res

return 0

```

- в качестве основы взята сортировка слиянием с сигнальными элементами
- смотрим насколько элемент переместился и добавляем к res

- ТЕСТЫ

```
import memory_profiler
import time
import unittest

from lab2.task3.src.main import merge_sort

def normVid(array: list) -> str:
    s = ""
    for i in range(len(array) - 1):
        s += str(array[i]) + " "
    s += str(array[-1])
    return s

class MergeSortTestCase(unittest.TestCase):

    def test_merge(self):
        file = open(
            "C:\\Users\\User\\PycharmProjects\\algorithms-and-
            data-structures\\lab2\\task3\\src\\input.txt",
            "w")
```

```

        file.write("10**5\n" + normVid([i for i in
range(10 ** 5, 0, -1)]))

    file = open(

"C:\\Users\\User\\PycharmProjects\\algorithms-and-
data-structures\\lab2\\task3\\src\\input.txt").rea
dlines()[

        1]

    array = list(map(int, file.split()))

    memory_before =
memory_profiler.memory_usage()[0]

    inversion_count = merge_sort(array, 0,
len(array))

    tmp = time.time()

    self.assertEqual(inversion_count,
(len(array) * (len(array) - 1)) // 2)

    tmp = time.time() - tmp

    memory_after =
memory_profiler.memory_usage()[0]

```

```

        print("Время выполнения: ", tmp)

        print("Использование памяти: ",
memory_after - memory_before, "МБ")

        with
open("C:\\Users\\User\\PycharmProjects\\algorithms
-and-data-structures\\lab2\\task3\\src\\output.txt
",

            "w") as file:
            file.write(str(inversion_count))

def test_merge_1000(self):
    N = 1000
    array = [i for i in range(N, 0, -1)]

    memory_before =
memory_profiler.memory_usage()[0]

    inversion_count = merge_sort(array, 0,
len(array))

    tmp = time.time()

    self.assertEqual(inversion_count,
(len(array) * (len(array) - 1)) // 2)

    tmp = time.time() - tmp

```

```

        memory_after =
memory_profiler.memory_usage()[0]

        print()
        print(N, inversion_count)
        print("Время выполнения: ", tmp)
        print("Использование памяти: ",
memory_after - memory_before, "МБ")

    def test_merge_10000(self):
        N = 10000
        array = [i for i in range(N, 0, -1)]

        memory_before =
memory_profiler.memory_usage()[0]

        inversion_count = merge_sort(array, 0,
len(array))

        tmp = time.time()
        self.assertEqual(inversion_count,
(len(array) * (len(array) - 1)) // 2)
        tmp = time.time() - tmp

```

```

        memory_after =
memory_profiler.memory_usage()[0]

        print()
        print(N, inversion_count)
        print("Время выполнения: ", tmp)
        print("Использование памяти: ",
memory_after - memory_before, "МБ")

    def test_merge_100000(self):
        N = 16
        array = [i for i in range(N, 0, -1)]

        memory_before =
memory_profiler.memory_usage()[0]

        inversion_count = merge_sort(array, 0,
len(array))

        tmp = time.time()
        self.assertEqual(inversion_count, 120)
        tmp = time.time() - tmp

        memory_after =
memory_profiler.memory_usage()[0]

```

```

print()
print(N, inversion_count)
print("Время выполнения: ", tmp)
print("Использование памяти: ",
memory_after - memory_before, "МБ")

if __name__ == '__main__':
    unittest.main()

```

ВВОД	Время	Память МВ	Вывод
10**5	0	0.5820312	4999950000
10**3	0	0.062	499500
16	0	0.0215429	120

Задание 4

В этой задаче вы реализуете алгоритм бинарного поиска, который позволяет очень эффективно искать (даже в огромных) списках при условии, что список отсортирован. Цель - реализация алгоритма двоичного (бинарного) поиска.

```
import random

def binSearch(array: list[int], number: int):
    left = 0
    right = len(array)

    while right-left > 1:
        center = (left + right) // 2
        if array[center] <= number:
            left = center
        else:
            right = center
    if array[left] == number:
        return left
    return -1
```

- стандартный бинарный поиск
- тесты


```

import memory_profiler
import random
import time
import unittest

from lab2.task4.src.main import binSearch

class testBinSearch(unittest.TestCase):

    def test_base(self):
        file = open(

"C:\\Users\\User\\PycharmProjects\\algorithms-and-
data-structures\\lab2\\task4\\src\\input.txt").rea
dlines()[
    1]
        array = list(map(int, file.split()))

        file = open(

"C:\\Users\\User\\PycharmProjects\\algorithms-and-
data-structures\\lab2\\task4\\src\\input.txt").rea
dlines()[

```

```

        3]

        values = list(map(int, file.split()))

        memory_before =
memory_profiler.memory_usage()[0]

        final_array = [binSearch(array, i) for i in
values]

        tmp = time.time()

        self.assertEqual(final_array, [2, 0, -1, 0,
-1])

        tmp = time.time() - tmp

        memory_after =
memory_profiler.memory_usage()[0]

        print()

        print("Время выполнения: ", tmp)

        print("Использование памяти: ",
memory_after - memory_before, "МБ")

    def test_100000(self):
        N = 10**5

        array = [i for i in range(0, N)]

```

```

        values = [i for i in range(0, N, 20)]

        memory_before =
memory_profiler.memory_usage()[0]

        final_array = [binSearch(array, i) for i in
values]

        tmp = time.time()

        self.assertEqual(final_array, values)

        tmp = time.time() - tmp

        memory_after =
memory_profiler.memory_usage()[0]

        print()

        print(N)

        print("Время выполнения: ", tmp)

        print("Использование памяти: ",
memory_after - memory_before, "МБ")

if __name__ == '__main__':
    unittest.main()

```

ввод	Время	Память МВ	Вывод
10**5	0	0.13671	0 20 40
пример	0	0.0	2 0 -1 0 -1

-

Задание 5

Правило большинства - это когда выбирается элемент, имеющий больше половины голосов. Допустим, есть последовательность A элементов $a_1, a_2, \dots, a_n$, и нужно проверить, содержит ли она элемент, который появляется больше, чем $n/2$ раз. Наивный метод это сделать:

```
import random
import time

def merge(array: list[int], left: int, center: int,
          right: int) -> list[int]:
    left_array = array[left:center]
    right_array = array[center:right]
    id_left = 0
    id_right = 0

    left_array.append(2 ** 62 - 1)
    right_array.append(2 ** 62 - 1)

    for key in range(left, right):
        if left_array[id_left] <=
right_array[id_right]:
            array[key] = left_array[id_left]
            id_left += 1
        else:
            array[key] = right_array[id_right]
```

```

        id_right += 1
    return array

def merge_sort(array: list[int], left, right) ->
    list[int]:
    if right - left != 1:
        center = (left + right) // 2

        merge_sort(array, left, center)
        merge_sort(array, center, right)

        merge(array, left, center, right)

    return array

def binSearch(array: list[int], number: int):
    left = 0
    right = len(array)

    while right - left > 1:
        center = (left + right) // 2
        if array[center] <= number:
            left = center
        else:
            right = center

```

```

    if array[left] == number:
        return left
    return -1

def majority(array: list[int]):
    sorted_array = merge_sort(array, 0, len(array))
    lenght = len(sorted_array)

    variable = sorted_array[lenght // 2]

    left, right = 0, 0
    for i in range(lenght):
        if sorted_array[i] == variable:
            left = i
            break
    for i in range(lenght - 1, -1, -1):
        if sorted_array[i] == variable:
            right = i
            break
    if right - left < lenght // 2:
        return 0
    return 1

def main():
    with open("input.txt") as file:

```

```

    array = list(map(int,
file.readlines()[1].split()))

    file.close()

with open("output.txt", "w") as file:
    file.write(str(majority(array)))

```

- 1) отсортируем за $n \log n$
- 2) если существует мажорити, то это центральный элемент
- 3) за n ищем левый конец и правый цепочки из мажорити
- 4) $\text{right} - \text{left} == \text{количество}$

длина массива	Время	Память	Вывод
100000	0.216000080108 64258	0.1328125 МБ	0
пример	0	0	1

Вывод:

Сортировка слиянием крайне эффективный алгоритм. Бинарный поиск тоже.